

Un instrument
d'aide à la rédaction
pour les documents
exprimant des
modèles logiciels.

Plan

1. Le choix du sujet (pourquoi)
2. L'importance des documents
3. La mauvaise qualité des documents (le problème)
4. Récapitulatif théorique
5. Récapitulatif du problème
6. L'instrument développé (la solution)
7. Évaluation de la solution (la qualité)
8. Conclusion

Le choix du sujet

D'abord, il y a :

- le désir d'obtenir des logiciels de plus grande qualité;
- les limites aux procédés de développement logiciel pour remplir cet objectif;
- les problèmes de qualité subsistants dans les documents;
- les répercussions engendrées par des documents de mauvaise qualité;
- la taille imposante du sujet qu'est la documentation informatique.

Ainsi, le problème est devenu celui de la formalisation et de la compréhension des documents servant aux modèles.

L'importance des documents

La documentation logicielle a pour fonction première de transmettre, au moyen de documents, des connaissances organisées en modèles sur lesquels toute opérationnalisation d'un traitement informatique est fondée.

1. l'informatique ne se limite pas au code source;
2. les documents font partie du procédé de développement;
3. les documents exprimant des modèles sont essentiels au développement.

Un sommaire de plusieurs documents existants

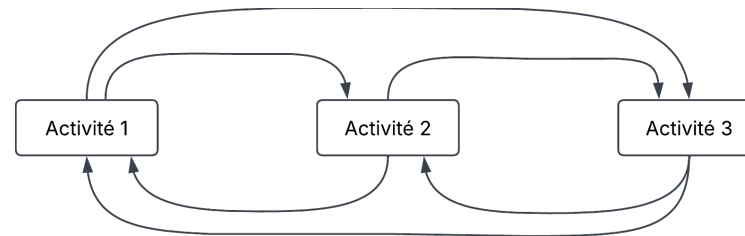
Standard \ Phase	Analyse	Conception	Concrétisation	Évaluation	Transition
<i>DID title</i> MIL-STD-498 (1994) [76]	• system/subsystem specification • interface requirements specification • software requirements specification	• system/subsystem design description • interface design description • software design description • database design description	Non applicable	• software test plan • software test description • software test report	• software installation plan • software transition plan • operational concept description • software product specification • software version description • software user manual • software input/output manual • software center operator manual • computer operation manual
<i>Documents</i> DO-178C (2012) [104, 116, 315]	• system requirements • software high-level requirements • specification models	• low-level requirements • software architecture • design models • derived requirement	• source code • build instructions	• software verification plan • software verification cases and procedures • software verification result	Non applicable
<i>Documents</i> IEC 61513:2013 [136, 363]	• requirements specification (functionality, performance) • functional assignment • system requirements specification • application functions requirements specification	• architectural design • system detailed design	• application software code • system integration plan • system integration • system integration test report • system integration report	• system validation documentation (system validation plan, system validation test report, system validation report)	• qualification plan (functional and environmental qualification, software evaluation and assessment) • system installation plan • system operation plan • system maintenance plan
<i>Information elements</i> ISO 15289:2011 [134]	• system requirements specification • software requirements specification	• system architecture description • interface description • system/software design description	• implementation procedure • integration procedure • test procedure • implementation report • test report	• verification plan • validation plan • verification procedure • validation procedure • verification report	• release plan • maintenance plan • maintenance procedure • user documentation

Explications sur le procédé

Définitions en français :

- **procédé** : ensemble structuré d'activités visant un but logiquement interrelié par des antécédents et des conséquents où des intrants sont transformés en extrants;
- **processus** : instance d'un procédé;
- **artefact** : intrant tangible ou extrant tangible à un processus.

Représentation de procédé sous la forme de diagramme :



Complément d'information sur le terme artefact en langage mathématique :

$$artefact \in \{document, entité informatique, \dots\}$$

$$artefact \in \{intranant, extrant\}$$

L'essentiel du procédé de développement

Lors de ce procédé, l'informatique appliquée va :

1. chercher à définir, à s'approprier, à circonscrire et à caractériser les solutions acceptables pour un problème en particulier;
2. réaliser et établir une entité informatique qui solutionne ce problème.

Un exemple

Exemple : calcul de la quantité de carburant

- Les documents comprennent :
 - besoin « Le système doit calculer la quantité de carburant nécessaire pour le restant du trajet. »;
 - descriptions du domaine avec différentes formules de physique concernant la gravité, la distance, la combustion, etc.;
 - exigences « Le calcul du carburant s'effectue comme X et les données prennent la forme d'Y. »;
- Un modèle du problème avec les concepts et les formules;
- Un modèle de solution avec une représentation des données et des règles de calcul du carburant et du trajet;
- Un logiciel, c'est-à-dire le programme qui effectue le calcul.

La mauvaise qualité des documents

Il y a une mauvaise compréhension des objectifs et de l'usage :

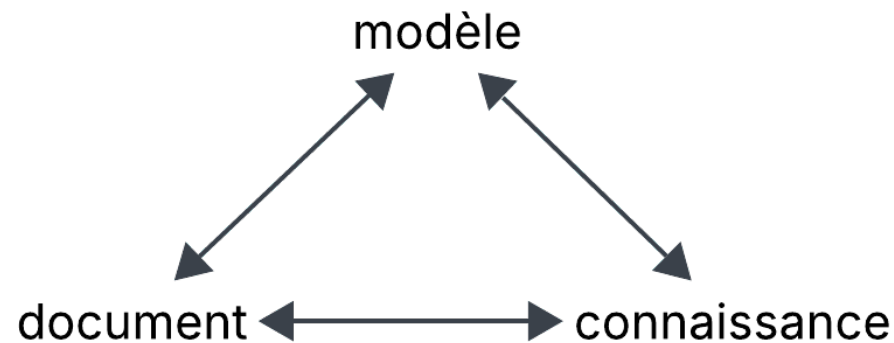
- des langages formels;
- des artefacts dont les documents font partie.

Cela conduit à :

- une augmentation de l'usage de langage non formel;
- une utilisation de mesures et de critères subjectifs;
- une perception d'un manque de cohérence, de couverture et d'efficacité des artefacts produits;
- potentiellement des répercussions négatives sur les modèles et les connaissances.

Des relations

Ce qui est contenu dans les documents et les modèles a généralement un impact visible ou non immédiatement visible dans les logiciels produits.



- **modèle** : représentation d'un sujet pour en permettre le traitement;
- **modèle formel** : est composé d'un langage formel et d'un ensemble de règles et vise la représentation d'un sujet pour en permettre le traitement;

La qualité

Pour des documents, des modèles ou des logiciels, il est possible de retrouver ces caractéristiques de qualité :

type	caractéristique
absolues	cohérence, couverture et efficacité
relatives	complétude, efficience et adaptabilité
universelles	réfutabilité et éthique

La cohérence

- définition de **système logique** : est composé d'un langage formel et d'un ensemble de règles (axiome, proposition, logique) permettant d'inférer.
- définition de **cohérence** : système logique où aucune contradiction ne s'est encore manifestée.

Les systèmes logiques permettent à un modèle formel ou système formel:

- de le rendre exécutable;
- d'y détecter des incohérences.

Les langages formels

Un langage formel comme le langage mathématique offrent :

1. un ensemble de symboles précis;
2. une syntaxe et une sémantique définies;
3. une maîtrise largement partagée sur la planète.

Plusieurs langages formels en informatique ont les inconvénients suivants :

1. de récupérer des symboles déjà existants;
2. d'avoir une syntaxe peu intuitive;
3. d'avoir une sémantique non définie;
4. d'avoir une maîtrise peu répandue.

Les langages non formels

La discipline qui traite ces langages est appelée **traitement automatique des langues**. Elle regroupe différentes techniques. Les grands modèles de langages sont un des résultats de ces techniques. Avec des inférences statistiques, elle offre des réponses satisfaisantes pour beaucoup d'utilisateurs.

Cependant, il y a des inconvénients :

1. elle ne surmonte pas les ambiguïtés des langages non formels;
2. la compréhension du fonctionnement interne de différentes techniques est très limitée;
3. la mise en place de certaines techniques nécessite énormément de ressources.

Récapitulatif théorique

Quelques constats :

- les documents exprimant des modèles sont nécessaires à la réalisation et l'établissement d'un logiciel pour modéliser/formaliser un problème et une solution;
- un logiciel concrétise un système formel ou un modèle formel;
- un système logique permet d'exécuter un système formel ou modèle formel et de trouver des incohérences;
- des documents erronés conduisent à des modèles erronés et des modèles erronés conduisent à des logiciels erronés.

Récapitulatif du problème

Sommairement les attentes de la solution devraient :

1. améliorer la compréhension des documents servant aux modèles;
2. rendre le travail avec les modèles plus efficaces;
3. être utilisable, peu importe l'endroit, les gens ou les procédés.

Sommairement le besoin de la solution doit permettre de manipuler, d'adapter, de réutiliser et d'évaluer les modèles exprimés à travers des documents. Cela permettra d'obtenir des documents, des modèles et des logiciels de meilleure qualité.

L'instrument développé

Sommairement, il permet :

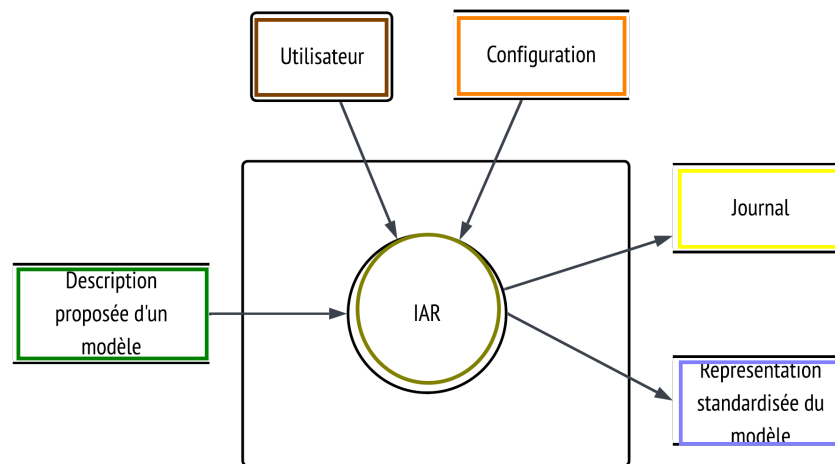
- de manipuler, d'adapter, de réutiliser et d'évaluer les modèles exprimés à travers des documents;
- de faciliter l'usage de langage formel;
- d'objectiver la qualité.

Deux groupes d'exigences :

- sur l'exploitation des modèles;
- sur l'exploitation des métamodèles.

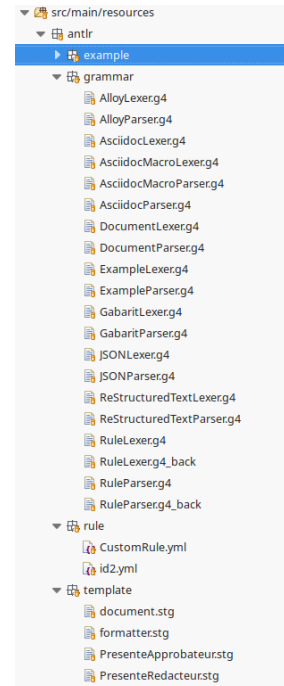
Explication sur le fonctionnement de l'IAR

- descriptions formelles exprimant des métamodèles ainsi que des règles (orange);
- documents définissant un modèle ou des changements à un modèle (vert);
- applique des transformations pour obtenir un modèle formel respectant un système logique (or);
- documents produits à partir d'un modèle et selon des règles (bleu).



Définir ses langages formels

- l'instrument permet d'employer les descriptions formelles souhaitées pour appliquer les différentes transformations;
- une extension à la preuve de concept est celle d'être à même de développer ses descriptions à l'intérieur de l'instrument;
- les langages formels permis actuellement sont la forme Backus-Naur étendue (épuration et extraction), le langage StringTemplate (présentation), langage de l'instrument Easy-Rules (vérification) et le langage Alloy (règles formelles du modèle).



Interface utilisateur

- sélectionner les transformations formelles (en rose);
- entrer les descriptions du modèle (en vert);
- observer une description standardisée du modèle (en mauve);
- observer le journal (en jaune).

[illegible]

L'évaluation de la solution

Il y a :

1. la vérification du respect des exigences avec des cas de tests;
2. la validation des attentes envisagées avec des expérimentations.

La vérification

1. il y a 5 cas de tests qui tentent de couvrir l'ensemble des 15 premières exigences;
2. ces cas de tests sont implémentés puis exécutés;
3. l'exécution permet de trouver des incohérences, de connaître l'efficacité et la couverture des exigences par l'implémentation.

La validation

1. consiste en l'ébauche d'un protocole expérimental;
2. vise à valider les critères de qualité que sont : la complétude, l'efficience et l'adaptabilité;
3. il s'agit d'une prochaine étape à ce travail.

Conclusion

Cet instrument d'aide à la rédaction contribue à :

- donner un instrument pour contrôler les documents ;
- se familiariser avec le concept de modèle, incluant les langages formels et les règles ;
- se réapproprier la création de certains modèles, comme des grammaires et des métamodèles ;
- évaluer la façon et l'impact de mettre en place de bonnes pratiques.

Cependant, la forme actuelle (preuve de concept) limite la validation possible de cette solution.